

День 2. Алгоритми та обчислювальна складність, частина 1

Складність алгоритмів, абстрактні типи даних, пошук, сортування та графи

Що таке алгоритм та його складність

Переходимо до розділу «Алгоритми та обчислювальна складність» — від восьми до дванадцяти відсотків балів. Це фундаментальна тема, тож розберемо її уважно. Почнемо з означень.

Алгоритм — це чітка скінченна послідовність дій, що приводить від вхідних даних до результату. Ключові вимоги до алгоритму: він має бути однозначним, скінченним, тобто завершуватися, та результативним.

Найважливіше поняття розділу — складність алгоритму. Вона буває двох видів. Часова складність показує, скільки часу, тобто скільки операцій, потребує алгоритм залежно від розміру вхідних даних. Просторова складність показує, скільки додаткової пам'яті він потребує. Зверни увагу: складність вимірюють не в секундах чи байтах, а в тому, як зростають витрати зі збільшенням обсягу даних.

Для опису складності використовують так звану O -нотацію, «велике O ». Вона показує характер зростання, відкидаючи константи й несуттєві доданки. Розберемо основні класи складності від найкращого до найгіршого, бо це треба добре відчувати.

Складність O від одиниці — константна. Час не залежить від обсягу даних узагалі. Наприклад, узяти елемент масиву за індексом — миттєво, скільки б елементів не було.

Складність O від логарифма n — логарифмічна. Час зростає дуже повільно: подвоєння даних додає лише один крок. Класичний приклад — двійковий пошук.

Складність O від n — лінійна. Час зростає прямо пропорційно даним. Удвічі більше даних — удвічі більше роботи. Наприклад, перебрати всі елементи один раз.

Складність O від n на логарифм n — характерна для добрих алгоритмів сортування. Трохи гірша за лінійну, але цілком прийнятна.

Складність O від n у квадраті — квадратична. Час зростає як квадрат обсягу. Удесятеро більше даних — у сто разів більше роботи. Типова для простих, наївних сортувань і для вкладених циклів, де кожен елемент порівнюється з кожним.

І нарешті експоненційна складність, наприклад O від двох у степені n . Час зростає катастрофічно швидко; такі алгоритми придатні лише для дуже малих даних. Запам'ятай ієрархію від найкращого до найгіршого: константна, логарифмічна, лінійна, n на логарифм n , квадратична, експоненційна.

Абстрактні типи даних

Тепер про абстрактні типи даних, скорочено АТД. Абстрактний тип даних — це опис структури через її поведінку, через набір операцій, незалежно від того, як вона реалізована всередині. Тобто ми говоримо, що структура вміє робити, а не як саме вона це робить. Розберемо основні.

Стек — це структура, що працює за принципом «останнім прийшов — першим пішов», англійською LIFO. Уяви стос тарілок: кладемо й

беремо лише зверху. Додавання й вилучення відбуваються з одного кінця.

Черга — структура за принципом «першим прийшов — першим пішов», англійською FIFO. Як черга в магазині: хто став першим, того й обслужать першим. Додаємо в один кінець, забираємо з іншого.

Черга з пріоритетами — особлива черга, де елементи обслуговуються не за порядком надходження, а за важливістю: спершу виходить елемент з найвищим пріоритетом.

Список — упорядкована послідовність елементів. Зв'язний список зберігає елементи як вузли, де кожен вузол містить дані й посилання на наступний. Однобічнозв'язний список має посилання лише вперед, на наступний елемент. Двобічнозв'язний список має посилання і вперед, і назад, що дозволяє рухатися в обидва боки.

Вектор, або масив — це послідовність елементів, що зберігаються порядку в пам'яті, з доступом за індексом. Доступ за індексом дуже швидкий, константний, але вставлення в середину повільне, бо доводиться зсувати елементи.

Множина — це сукупність унікальних елементів без повторень і без визначеного порядку. Мультимножина дозволяє повторення, тобто зберігає, скільки разів елемент трапляється. Словник, він же асоціативний масив чи відображення, зберігає пари «ключ — значення» і дозволяє швидко знаходити значення за ключем. КORTEЖ — це впорядкований незмінний набір елементів фіксованого розміру.

Важлива ідея: різні реалізації одного й того самого АТД дають різну складність операцій. Наприклад, словник на основі хеш-таблиці шукає значення за ключем у середньому за константний час, а той самий словник на основі дерева — за логарифмічний. Тому на іспиті можуть

запитати, яка складність тієї чи іншої операції для конкретної реалізації.

Пошук і сортування

Тепер базові алгоритми. Спершу пошук. Лінійний пошук послідовно перебирає елементи, доки не знайде потрібний. Його складність лінійна, O від n , і він працює на будь-яких даних. Двійковий пошук набагато швидший, логарифмічний, O від логарифма n , але має важливу умову: дані мають бути попередньо відсортовані. Він щоразу ділить діапазон пошуку навпіл, відкидаючи половину. Запам'ятай цю умову — двійковий пошук тільки на відсортованих даних.

Тепер сортування. Його ділять на прості й удосконалені. Прості сортування мають квадратичну складність, O від n у квадраті, і їх три.

Сортування вибором: на кожному кроці знаходимо найменший елемент серед тих, що лишилися, і ставимо його на місце.

Сортування вставками: беремо елементи по черзі й вставляємо кожен у вже відсортовану частину на правильне місце. На майже відсортованих даних воно працює близько до лінійного — це його перевага.

Сортування обмінами, відоме як «бульбашкове»: сусідні елементи порівнюються й міняються місцями, доки весь масив не впорядкується; найбільші елементи поступово «спливають» угору.

Удосконалені сортування швидші, з середньою складністю O від n на логарифм n . Сортування деревом, або пірамідальне сортування, використовує структуру купи, щоб щоразу швидко витягати найбільший елемент. Сортування Шелла — це покращене сортування вставками, де спочатку порівнюють далекі елементи, поступово зменшуючи відстань; воно швидше за прості. Швидке сортування,

англійською quicksort, обирає опорний елемент і ділить масив на менші й більші за нього, потім рекурсивно сортує частини; у середньому воно дуже швидке, n на логарифм n , хоча в найгіршому випадку може деградувати до квадратичного.

Загальне правило для іспиту: прості сортування — квадратичні, удосконалені — n на логарифм n . Двійковий пошук — логарифмічний, лінійний — лінійний.

Алгоритми на графах

І завершуємо графовими алгоритмами. Граф — це вершини, з'єднані ребрами; про самі графи детально буде в розділі дискретної математики, тут — алгоритми на них.

Два базові способи обходу графа. Пошук вшир, англійською BFS, досліджує граф «колами»: спершу всіх сусідів стартової вершини, потім сусідів сусідів, і так далі, рівень за рівнем. Він використовує чергу. Пошук вглиб, англійською DFS, навпаки, заглиблюється якомога далі одним шляхом, а коли впирається — повертається й пробує інший. Він використовує стек або рекурсію. Запам'ятай: пошук вшир — по рівнях, через чергу; пошук вглиб — углиб, через стек.

На основі обходу розв'язують інші задачі. Пошук зв'язних компонентів визначає, які групи вершин з'єднані між собою, а які ізольовані. Побудова кістякового дерева, англійською spanning tree, виділяє з графа дерево, що з'єднує всі вершини без зайвих ребер і без циклів.

Окрема важлива група — найкоротші шляхи. Задача пошуку найкоротших шляхів з виділеної вершини знаходить найкоротші відстані від однієї стартової вершини до всіх інших. Задача пошуку найкоротшого шляху між двома конкретними вершинами знаходить

оптимальний маршрут між заданою парою. Це фундаментальні задачі, що лежать в основі, наприклад, навігаційних систем.

Підсумок

Підсумуємо першу частину алгоритмів. Складність буває часова й просторова, її описують O -нотацією; запам'ятай ієрархію зростання від константної до експоненційної. Абстрактні типи даних описують структуру через її операції: стек — це LIFO, черга — FIFO, словник зберігає пари «ключ-значення». Прості сортування квадратичні, удосконалені мають складність n на логарифм n , а двійковий пошук логарифмічний, але лише на відсортованих даних. На графах пошук вшир іде по рівнях через чергу, а пошук вглиб заглиблюється через стек.

У другій частині розберемо стратегії розроблення алгоритмів — «розділяй і володарюй», динамічне програмування — та моделі обчислень.